

Linux Guest Customization & Network Injection

Offline Pre-Boot Configuration for Every Linux Distribution

Inject network configs (systemd-networkd, NetworkManager, netplan), user accounts, SSH keys, firstboot scripts, hostname, services, cloud-init seeds, custom initramfs drivers, and serial console — all applied offline before the VM ever boots on KVM. No guest agent required.

7 Injectors | systemd-networkd + NM | Users + SSH | Firstboot Scripts | cloud-init | 15+ Distros

Injection Architecture

Seven independent injectors applied offline in sequence — deterministic, pre-boot, no guest agent needed.



Injection Execution Order

#	Injector	CLI Flag	What It Does
1	Cloud-Init	<code>--cloud-init-config</code>	Drop-in cloud config + NoCloud seed (user-data, meta-data, network-config)
2	Firstboot Scripts	<code>--firstboot-scripts</code>	Systemd oneshot service + ordered script execution on first boot
3	Network Config	<code>--network-config-inject</code>	systemd-networkd .network/.netdev files or NetworkManager .nmconnection profiles
4	User Config	<code>--user-config-inject</code>	User creation, passwords, SSH keys, sudo, group membership
5	Service Config	<code>--service-config-inject</code>	Enable/disable/mask systemd services before boot
6	Hostname	<code>--hostname-config-inject</code>	Set /etc/hostname + /etc/hosts entries with FQDN support
7	initramfs Drivers	<code>--initramfs-add-drivers</code>	Add custom kernel modules to initramfs (NVMe, iSCSI, etc.)

Convenience Shortcuts

Flag	Effect
<code>--root-password 'pass'</code>	Sets root password (SHA-512 hashed, written to /etc/shadow)
<code>--ssh-authorized-key ~/.ssh/id.pub</code>	Deploys SSH key to root's authorized_keys (0600 perms)
<code>--serial-console</code>	Injects <code>console=ttyS0,115200</code> into GRUB + enables serial-getty@ttyS0
<code>--remove-vmware-tools</code>	Removes open-vm-tools package and vmtoolsd.service

All injections are offline — the disk is mounted via qemu-nbd, files are written directly, then unmounted. No guest boot, no SSH, no guest agent. Works on any Linux distro with any init system. Supports `--dry-run` for testing.

Network Configuration **Injection**

Pre-configure networking for KVM before the VM boots — systemd-networkd, NetworkManager, or both.

systemd-networkd

- **.network files:** Interface config (static IP, DHCP, DNS, routes)
- **.netdev files:** Virtual devices (bonds, bridges, VLANs, VXLANs, tunnels, macvlan)
- Configurable file priority (10-, 50-, 99-)
- Written to `/etc/systemd/network/`
- Optional: enable `systemd-networkd.service`
- Best for: RHEL 9+, Fedora, Photon OS, CoreOS

NetworkManager

- **.nmconnection files:** Full connection profiles
- Written to `/etc/NetworkManager/system-connections/`
- Auto `chmod 0600` (NM requirement)
- Optional: enable `NetworkManager.service`
- Best for: RHEL 8, Ubuntu Desktop, Fedora Workstation
- Supports bonds, bridges, VLANs, WiFi

Example: Static IP with Bond (systemd-networkd)

```
# network-config.yaml (passed via --network-config-inject)
networkd_units:
  - name: 10-bond0.netdev
    content: |
      [NetDev]
      Name=bond0
      Kind=bond
      [Bond]
      Mode=802.3ad
      MIIMonitorSec=100ms

  - name: 20-eth0.network
    content: |
      [Match]
      Name=eth0
      [Network]
      Bond=bond0

  - name: 20-eth1.network
    content: |
      [Match]
      Name=eth1
      [Network]
      Bond=bond0

  - name: 30-bond0.network
    content: |
      [Match]
      Name=bond0
      [Network]
```

```
Address=10.0.1.50/24
Gateway=10.0.1.1
DNS=10.0.1.53
DNS=1.1.1.1
```

```
enable_networkd: true
```

Example: VLAN + Bridge (NetworkManager)

```
# nm-config.yaml
nm_connections:
- filename: eth0.nmconnection
  content: |
    [connection]
    id=eth0
    type=ethernet
    interface-name=eth0
    [ipv4]
    method=auto

- filename: vlan100.nmconnection
  content: |
    [connection]
    id=vlan100
    type=vlan
    [vlan]
    parent=eth0
    id=100
    [ipv4]
    method=manual
    addresses=172.16.100.50/24
    gateway=172.16.100.1
    dns=172.16.100.53

enable_network_manager: true
```

User, SSH & Security Injection

Create users, deploy SSH keys, set passwords, configure sudo — all offline before first boot.

User Creation

- Username, UID, GID specification
- Group membership (multiple groups)
- Shell selection (default /bin/bash)
- Home directory with ownership
- GECOS/comment field
- Primary: useradd command
- Fallback: direct /etc/passwd editing

Password Management

- Plaintext auto-hashed (SHA-512)
- Pre-hashed password support
- Direct /etc/shadow editing
- Password age tracking
- Account lock/disable (! prefix)
- Root password shortcut flag

SSH Key Deployment

- ~/.ssh/authorized_keys creation
- Auto dir creation (0700)
- Auto file perms (0600)
- Ownership correction
- Multiple keys per user
- Root key shortcut flag

Example: Full User Configuration

```
# users.yaml (passed via --user-config-inject)
users:
  - username: admin
    uid: 1001
    groups: [wheel, docker]
    shell: /bin/bash
    password: "SecurePass123!"          # auto SHA-512 hashed
    ssh_authorized_keys:
      - "ssh-ed25519 AAAAC3... admin@corp"
      - "ssh-rsa AAAAB3... admin-backup@corp"
    sudo: "ALL=(ALL) NOPASSWD:ALL"

  - username: deploy
    uid: 1002
    groups: [docker]
    shell: /bin/bash
    ssh_authorized_keys:
      - "ssh-ed25519 AAAAC3... deploy@ci"
    sudo: "ALL=(ALL) NOPASSWD:/usr/bin/systemctl, /usr/bin/docker"

  - username: monitoring
    uid: 1003
    groups: [adm]
    password_hash: "$6$rounds=5000$salt$hash..." # pre-hashed

disable_users:
  - games
  - ftp

# Files written offline:
```

```
# /etc/passwd, /etc/shadow, /etc/group
# /home/admin/.ssh/authorized_keys (0600)
# /etc/sudoers.d/admin (0440)
```

Firstboot Script Injection

How It Works

- Scripts staged in `/usr/local/lib/hyper2kvm-firstboot/`
- Systemd oneshot service created & enabled
- Runs after `network.target` by default
- Ordered execution (lower order = earlier)
- Logged to `/var/log/hyper2kvm-firstboot.log`
- Auto-disables after first run (unless `keep_enabled`)
- Auto-adds shebang if missing

Example

```
# firstboot.yaml
service:
  description: "Post-migration setup"
  after: network-online.target
  environment:
    MIGRATION_ID: "batch-2026-03"
scripts:
  - name: 01-register-dns.sh
    order: 1
    content: |
      #!/bin/bash
      nsupdate -k /etc/rndc.key <
```

Service, Hostname & Boot Config

Control systemd services, set identity, customize initramfs drivers, enable serial console.

Service Configuration

```
# services.yaml
enable:
- sshd
- firewalld
- chronyd
- zabbix-agent

disable:
- bluetooth
- cups
- avahi-daemon

mask:
- NetworkManager-wait-online
- kdump
```

Creates/removes symlinks in `/etc/systemd/system/multi-user.target.wants/`. Mask creates symlink to `/dev/null`.

Hostname Configuration

```
# hostname.yaml
hostname: web-prod-01
domain: corp.example.com

# Custom /etc/hosts entries
hosts:
"10.0.1.10": "db-primary.corp.example.com db-primary"
"10.0.1.11": "db-replica.corp.example.com db-replica"
"10.0.1.20": "cache-01.corp.example.com cache-01"
```

Updates `/etc/hostname` + `/etc/hosts` with FQDN on 127.0.1.1. Preserves existing localhost entries.

initramfs Custom Drivers

- Add hardware-specific kernel modules to initramfs
- Written to `/etc/dracut.conf.d/hyper2kvm.conf`
- Deduplication (removes duplicates, preserves order)
- Validates modules exist in kernel
- Works with dracut (RHEL/Fedora/SUSE) and update-initramfs (Debian/Ubuntu)

```
# Default VirtIO modules (always included):
# virtio virtio_ring virtio_blk virtio_scsi
# virtio_net virtio_pci nvme ahci sd_mod

# Add custom drivers:
initramfs_add_drivers: "nvme e1000e mlx5_core ixgbe dm_crypt xts"

# CLI:
h2kvmctl --initramfs-add-drivers "nvme e1000e mlx5_core"
```

Serial Console

- Injects `console=ttyS0,115200` + `console=tty0`
- Updates GRUB_CMDLINE_LINUX in `/etc/default/grub`
- Regenerates `grub.cfg`
- Enables `serial-getty@ttyS0.service`
- Supports GRUB2, GRUB Legacy, LILO, extlinux
- BLS (Boot Loader Specification) entries supported
- Checks for existing entries (no duplicates)
- Enabled by default (`--no-serial-console` to disable)

```
# Enables: virsh console
serial_console: true # default

# Result in /etc/default/grub:
GRUB_CMDLINE_LINUX="... console=ttyS0,115200 console=tty0"
```

Cloud-Init Seed Injection

Drop-in Config

```
# cloud-init.yaml
cloud_config:
  hostname: web-prod-01
  manage_etc_hosts: true
  ssh_pwauth: true
  users:
    - name: admin
      groups: [wheel]
      sudo: ALL=(ALL) NOPASSWD:ALL
      ssh_authorized_keys:
        - ssh-ed25519 AAAAC3...
  packages:
    - htop
    - vim
  runcmd:
    - systemctl enable --now httpd
```

Written to `/etc/cloud/cloud.cfg.d/99-hyper2kvm.cfg`

NoCloud Seed

```
# cloud-init.yaml (with NoCloud seed)
nocloud_seed:
  user_data: |
    #cloud-config
    hostname: web-prod-01
    users:
      - name: admin
        ssh_authorized_keys:
          - ssh-ed25519 AAAAC3...
  meta_data: |
    instance-id: i-hyper2kvm-001
    local-hostname: web-prod-01
  network_config: |
    version: 2
    ethernets:
      eth0:
        dhcp4: true
  datasource_hint: NoCloud
```

Seed at `/var/lib/cloud/seed/nocloud-net/`

Complete Guest Customization — Before First Boot

Network (systemd-networkd + NM) | Users + SSH Keys + Sudo | Firstboot Scripts | Services | Hostname | Cloud-Init | initramfs | Serial Console

All offline. All deterministic. All 15+ distros. No guest agent. No cloud metadata.